StreamForge: A Cost-Aware, Event-Driven Architecture for Cloud Video Transcoding and Delivery with Scale-to-Zero Workers

Kartika Dhonde
Department of Computer Engineering
Mukesh Patel School of Technology Management and Engineering
NMIMS University
Mumbai, India
kartika.dhonde16@nmims.in

Kshitij Jhalak
Department of Computer Engineering
Mukesh Patel School of Technology Management and Engineering
NMIMS University
Mumbai, India
kshitij.jhalak90@nmims.in

Shlok Doshi
Department of Computer Engineering
Mukesh Patel School of Technology Management and Engineering
NMIMS University
Mumbai, India
shlok.doshi49@nmims.in

Samar Kamat
Department of Computer Engineering
Mukesh Patel School of Technology Management and Engineering
NMIMS University
Mumbai, India
samar.kamat06@nmims.in

Rama Varshney
Department of Computer Engineering
Mukesh Patel School of Technology Management and Engineering
NMIMS University
Mumbai, India
rama.varshney@nmims.edu

***Abstract*—Video processing workloads are bursty and compute-heavy, which makes them expensive to host on always-on servers and awkward to run on function-as-a-service platforms with short execution limits. This paper presents StreamForge, an open-source platform on Amazon Web Services (AWS) that ingests user uploads, transcodes them into 720p and 480p H.264 renditions with a JPEG thumbnail, and delivers them through a content delivery network. Browsers upload directly to object storage through presigned URLs, so the control-plane API never carries media bytes. Storage events feed a message queue whose depth drives a containerized FFmpeg worker fleet on Fargate Spot that scales between zero and three tasks. A dead-letter queue bounds the cost of malformed inputs. The entire stack (53 Terraform resource blocks) is declared as code and is deployed by a GitHub Actions pipeline that uses OpenID Connect federation instead of stored credentials, blocks images with fixable critical vulnerabilities, and detects automatic rollbacks by the ECS deployment circuit breaker. We benchmark the production worker image under a 1-vCPU, 2-GB resource limit matching the deployed task size and find that a 1080p clip takes about 1.53 s of processing per second of video, that the two renditions together are about 86% smaller than the 8 Mb/s source, and that the 1-vCPU size is inefficient: a 2-vCPU limit is 2.6× faster. We also present a cost model showing that, with no NAT gateway and an idle worker cost of zero, the always-on footprint reduces to one load balancer and one small API task.**

***Index Terms*—cloud computing, video transcoding, event-driven architecture, serverless containers, autoscaling, infrastructure as code, CI/CD, DevSecOps**

I. Introduction

Video dominates consumer internet traffic [1], and nearly every application that accepts user video must convert uploads into a small set of standard renditions before it can stream them. Transcoding is CPU-bound, takes time proportional to clip duration, and arrives in bursts. Running it on dedicated virtual machines means paying for idle capacity between bursts. Running it on function-as-a-service (FaaS) platforms avoids idle cost [2] but runs into per-invocation duration, memory and ephemeral-storage limits [3], and full-length transcodes do not fit them without restructuring the encoder [4], [5].

There is a second, less discussed cost problem. Reference cloud architectures often place compute in private subnets behind managed NAT gateways, provision databases, and run several CDN distributions. For student, prototype and small-team budgets, these fixed costs can exceed the variable cost of the workload itself.

This paper describes StreamForge, a video upload, transcoding and streaming platform designed to keep fixed cost low while retaining the operational properties expected of production systems: elastic processing, bounded failure cost, least-privilege access, reproducible infrastructure, and automated, verifiable deployment. Its contributions are:

1) An event-driven architecture in which media bytes never pass through the API tier, with worker capacity derived from queue depth and scaled to zero when idle (Section III).
2) A failure model built on at-least-once queue delivery, idempotent output keys and a dead-letter queue, with an explicit analysis of Spot interruption and duplicate delivery (Sections III-B and VI).
3) A fully declarative deployment and a delivery pipeline

that uses keyless OIDC federation, image vulnerability gating and circuit-breaker rollback detection (Section IV).
4) A measurement of the production worker image under the deployed resource limits, and a cost model of the fixed and per-job costs (Section V).

The source code, infrastructure definitions and workflows are publicly available [19].

II. Related Work

Serverless video processing. ExCamera [4] showed that splitting a video into small chunks and encoding them across thousands of short-lived functions can cut latency by orders of magnitude, at the cost of a custom encoder that passes state between chunks. Sprocket [5] generalized this into a serverless video processing framework with a pipeline abstraction. Both target latency for long videos. StreamForge targets cost and operational simplicity for short user-generated clips: it runs an unmodified FFmpeg [6] per job inside a container, which avoids FaaS duration limits without changing the encoder.

Encoding and streaming. H.264/AVC [7] remains the most widely decodable codec. Production services choose encoding ladders per title [8] and stream them with adaptive protocols such as MPEG-DASH [9]. StreamForge uses a fixed two-rung ladder with progressive MP4 download and manual resolution selection. Section VII discusses moving to segmented adaptive streaming.

Cluster management and delivery practice. Container orchestration grew out of large-scale cluster managers [10]. StreamForge uses a managed serverless container service, so it does not operate a cluster. Its pipeline follows continuous delivery practice [11], infrastructure-as-code principles [12], and container security guidance [13]. Its rollback behavior applies the circuit-breaker pattern [14] at the deployment level.

III. System Design

Fig. 1 shows the architecture. It separates a *control plane* (API and metadata store), an *asynchronous processing plane* (queue, autoscaler and workers), and a *delivery plane* (object storage and CDN). The planes communicate only through durable managed services, so each can fail or scale independently.

A. Ingestion with Presigned Uploads

A client registers a video with POST /api/videos and a title. The API assigns a UUIDv4 identifier, writes a metadata item with status uploaded, and returns a presigned S3 PUT URL for the key raw/{id}.mp4, valid for 15 minutes and bound to content type video/mp4. The browser then uploads the file directly to S3 with XMLHttpRequest, which reports upload progress (the Fetch API does not).

Because the API never receives media bytes, its memory footprint and connection lifetime are independent of upload size. The deployed API task has 0.25 vCPU and 512 MB of memory. The uploads bucket accepts cross-origin PUT only, blocks all public access, expires raw objects after three days, and aborts incomplete multipart uploads after one day.

TABLE I
Encoding Ladder (Common: libx264, AAC, +faststart)

Output	Scale filter	Rate control	Preset
720p MP4	scale=-2:720	CRF 28	veryfast
480p MP4	scale=-2:480	CRF 28	veryfast
Thumbnail	scale=640:-1	1 frame at 1 s	–

B. Event Queue and Failure Semantics

S3 publishes ObjectCreated events whose keys match raw/*.mp4 to the SQS queue video-jobs. A queue policy accepts messages only from that bucket and account. The queue uses a 300-second visibility timeout, 20-second long polling, and a redrive policy with maxReceiveCount = 3 to a dead-letter queue (DLQ) that retains messages for 14 days.

Workers delete a message only after every output is uploaded and the metadata item is marked ready. On any exception the worker marks the item failed, records a truncated error, and leaves the message in the queue. SQS redelivers it once the visibility timeout expires, and after three receives moves it to the DLQ. A CloudWatch alarm notifies operators when the DLQ is not empty, and the console's redrive action returns messages to the main queue once the cause is fixed. This bounds the compute that a corrupt or adversarial upload can consume to three attempts.

Delivery is at-least-once. The worker skips any job whose item is already ready, and all output keys are derived deterministically from the video identifier (processed/{id}/720p.mp4, processed/{id}/480p.mp4, thumbnails/{id}/thumb.jpg). A duplicate that arrives during processing therefore overwrites identical objects instead of creating divergent state.

C. Transcoding Worker

The worker is a Python 3.11 process in a multi-stage container image with FFmpeg 7.1, running as an unprivileged user (UID 10001). It loops on a single-message long poll. For each job it downloads the source to a per-job scratch directory and runs three FFmpeg invocations (Table I). It then uploads the outputs with explicit content types, updates the item, and removes the scratch directory. Each FFmpeg call has a timeout, and errors include the last lines of FFmpeg's standard error. Clips shorter than one second have no frame at 1 s, so the thumbnail step falls back to the first frame.

The -2 width constraint preserves aspect ratio and keeps the width even, as yuv420p requires. The +faststart flag moves the MP4 index to the start of the file so that playback can begin before the download completes. The worker handles SIGTERM by finishing the current job and then exiting. The task definition sets a 120-second stop timeout to match the two-minute Fargate Spot interruption notice.

D. Metadata and State Machine

Metadata lives in an on-demand DynamoDB table keyed by video_id. Each item carries the title, source key, output map, thumbnail key, attempt counter, last error, and timestamps. Its

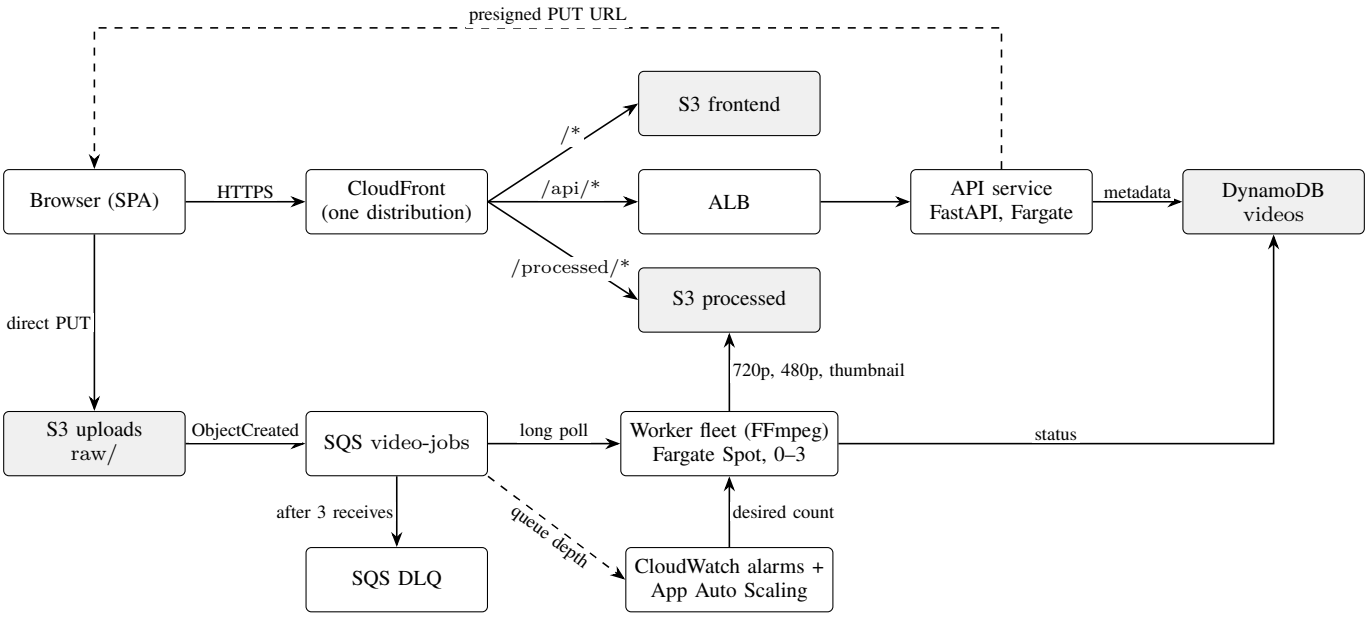


Fig. 1. StreamForge architecture. The API only handles metadata. Media goes browser → S3 uploads → worker → S3 processed → CloudFront.

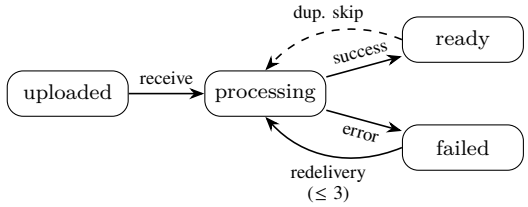


Fig. 2. Video status state machine. Redelivery after failure is bounded by the DLQ. A duplicate delivery of a ready video is acknowledged without work.

status follows the state machine in Fig. 2. The worker increments attempts atomically with ADD on every processing start, so operators can tell first-attempt failures from repeated ones.

E. Queue-Driven Autoscaling

Worker capacity is set by Application Auto Scaling with step policies triggered by two CloudWatch alarms:

- *Scale-out:* when the maximum ApproximateNumberOfMessagesVisible over a 60-second period is at least 1, add 1, 2 or 3 tasks for backlogs of 1–4, 5–9 or ≥ 10 messages, capped at 3 tasks, with a 120-second cooldown.
- *Scale-in:* when visible plus in-flight messages stay at 0 for five consecutive 60-second periods, set the desired count to 0.

Including in-flight (NotVisible) messages in the scale-in expression prevents the policy from stopping a worker in the middle of a long transcode. SQS stops publishing metrics for idle queues, so the scale-in alarm treats missing data as breaching (empty), and the scale-out alarm treats it as not breaching. Workers run on the FARGATE_SPOT capacity provider with 1 vCPU and 2 GB each.

F. Delivery

A single CloudFront distribution serves three origins over HTTPS: the frontend bucket (default behavior), the processed-media bucket (/processed/*, /thumbnails/*), and the API load balancer (/api/*, caching disabled, all methods forwarded). Both buckets are private and readable only by this distribution through Origin Access Control. Routing the API through the same distribution has two effects. The page and API share an origin, so the browser never makes a mixed-content request to the HTTP-only load balancer. And CORS is needed only for the direct upload to S3. HTTP 403 and 404 responses are rewritten to /index.html to support client-side routing.

IV. Implementation and Delivery Pipeline

A. Infrastructure as Code

All cloud resources are declared in 12 Terraform files that define 53 resource blocks, with remote state in a versioned S3 bucket. Table II summarizes them. ECS tasks run in two public subnets across two availability zones with public IPs and *no NAT gateway*. Security groups admit inbound traffic only from the internet to the load balancer on port 80, and from the load balancer's security group to the API on port 8000. Workers accept no inbound traffic at all. Buckets and registries are force-deleted on destroy, so a full teardown finishes in a single command, and CI discovers resource identifiers by name on every run instead of storing them. The result is that a destroy/apply cycle needs no manual reconfiguration.

Least privilege. The API task role can only put, get, update and scan items in the videos table and put objects under the raw/ prefix of the uploads bucket. That is the minimum needed to sign upload URLs. The worker role can receive, delete and change visibility of messages in the jobs queue, get

TABLE II
TERRAFORM RESOURCES BY CONCERN

File	Res.	Purpose
network.tf	8	VPC, 2 public subnets, IGW, SGs
s3.tf	7	3 buckets, CORS, lifecycle, events
sqs.tf	4	jobs queue, DLQ, redrive, policy
dynamodb.tf	1	on-demand videos table
ecr.tf	2	image lifecycle (keep 5)
iam.tf	10	OIDC provider, 4 least-privilege roles
ecs.tf	10	cluster, ALB, 2 services, 2 task defs
autoscaling.tf	9	scaling target, policies, alarms, SNS
cloudfront.tf	2	distribution, OAC
Total	53	

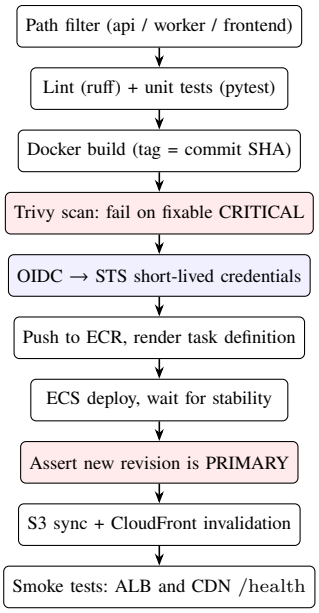


Fig. 3. CI/CD pipeline. Shaded stages are gates that fail the run. Credentials are requested only after the image scan passes.

and update items, get from the uploads bucket, and put to the processed bucket. Neither role can delete media or reach the other service's resources.

B. Continuous Integration and Deployment

Fig. 3 shows the deployment workflow, which runs on every push to main.

Change detection. A path filter builds and deploys only the services whose files changed. Manual dispatch rebuilds everything, which is used after recreating the stack.

Supply-chain gating. Each image is scanned with a pinned Trivy release. HIGH and CRITICAL findings are reported, and any *fixable* CRITICAL finding fails the build. The scanner runs before AWS credentials are loaded into the job, so a compromised scanner image cannot read them.

Keyless authentication. The pipeline obtains short-lived AWS credentials by exchanging its GitHub OIDC token [15] with STS. The role trust policy pins the audience to sts.amazonaws.com and restricts the subject claim to this repository, so no long-lived access key is stored in the repository's secrets. Section VI notes a weakness in one of the subject patterns.

Rollback detection. The API service enables the ECS deployment circuit breaker with automatic rollback. A rolled-back deployment still reaches a stable state, just on the *old* revision, so a plain stability wait reports success. The pipeline therefore also asserts that the newly registered task definition is the service's primary deployment and fails the run otherwise. A separate manually triggered workflow redeploys any earlier task-definition revision of either service. A third workflow runs terraform fmt, validate and plan on pull requests that touch infrastructure.

C. Local Emulation

A Docker Compose environment runs the API, worker and an nginx-served frontend against LocalStack, which emulates S3, SQS and DynamoDB. An initialization script creates the same buckets, queues, redrive policy, bucket notification and table. The whole pipeline can therefore be developed and demonstrated with no cloud account.

D. Client

The frontend is a dependency-free single-page application that lists videos, filters by status, and polls GET /api/videos every five seconds. It plays either rendition and switches resolution in place, restoring currentTime after the source changes.

V. EVALUATION

A. Methodology

We measured the production worker image, not a separate build, to isolate the dominant per-job cost: transcoding time. The image (Debian, FFmpeg 7.1.5, Python 3.11) ran under Docker with –cpus=1 –memory=2g, matching the deployed worker task, and with –cpus=2 for comparison. The harness called the worker's own command builders (rendition_command, thumbnail_command) and timed each FFmpeg invocation. Inputs were synthetic 1920×1080, 30 fps H.264 clips at 8 Mb/s with 48 kHz AAC audio, lasting 15, 30, 60 and 120 s. Each clip contains a moving test pattern with temporal noise, so that the encoder cannot exploit static content. Each configuration ran three times, and we report the mean and sample standard deviation. Section VI-A discusses how far these results transfer to Fargate.

B. Transcoding Time

Table III and Fig. 4 report the results. At 1 vCPU, total processing time grows linearly with clip duration. A least-squares fit over all runs gives $T \approx 1.53\,d - 1.2$ s, where d is the clip duration in seconds, so the worker needs about 1.53 s of compute per second of 1080p input. The 720p rendition takes 57% of the total and the thumbnail under one second. Doubling the CPU limit to 2 vCPU yields a 2.63–2.68× speedup, which is *superlinear*. The likely cause is thread oversubscription: FFmpeg sizes its libx264 thread pool from

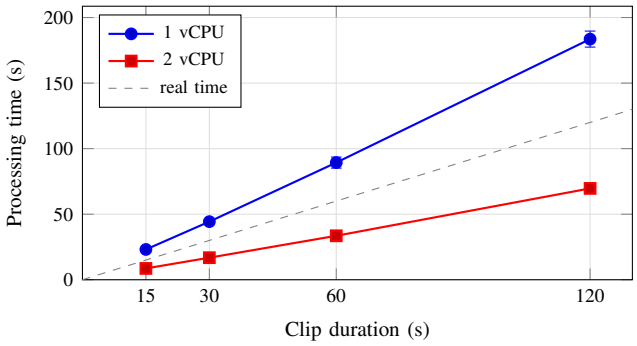


Fig. 4. Total worker processing time (720p, 480p and thumbnail) against clip duration. Error bars show one standard deviation over three runs.

the number of visible host cores, while the container is limited to one CPU's worth of time by a CFS quota, so many threads compete for a small budget and are throttled. If this effect also appears on Fargate, a 2-vCPU, 4-GB worker, which costs twice as much per hour, would finish jobs 2.6× faster and cost about 25% less per job. Setting `-threads` to match the task's vCPU count is a second, independent fix. Both need to be confirmed on Fargate itself (Section VI-A).

C. Output Size

Table IV lists output sizes. Both renditions together are 85.6–86.0% smaller than the 8 Mb/s source. Output bitrates are almost constant across durations, as expected from constant-quality (CRF) rate control on uniform content.

D. Functional Verification

The repository has 17 unit tests (6 for the API, 11 for the worker) that mock the AWS SDK. They cover the health endpoint on both route prefixes, video registration and presigned URL generation, rejection of empty titles, listing order and CloudFront URL attachment, unknown identifiers, S3 event parsing (test events, foreign prefixes and URL-encoded keys), FFmpeg command construction, the duplicate-delivery skip, the success path, and the failure path that keeps the message on the queue. All 17 pass under Python 3.11, and both services pass the `ruff` linter. The CI pipeline runs the same commands.

E. Cloud Deployment Measurements

Table V lists end-to-end figures for the live deployment in `ap-south-1`.

The configuration sets a lower bound on cold-start latency. The scale-out alarm evaluates one 60-second period of an SQS metric that is published at one-minute granularity. A job arriving on an idle system therefore waits for the next metric datapoint and alarm evaluation, and then for a Fargate task to pull its image and start, before a worker polls the queue. A warm worker's 20-second long poll returns as soon as a message is visible.

F. Cost Model

Table VI separates fixed hourly costs from per-job costs. We use published on-demand list prices [16]–[18] for `us-east-1`, the most commonly quoted region. Fargate Spot is discounted by up to 70% from these prices [16]. Prices vary by region and change over time, so the table is for comparison only.

While deployed, the fixed cost is about $0.035/h, or about $0.84/day. A conventional design with one NAT gateway and one always-on worker on the same instance sizes adds about $0.094/h on top of that, before data processing charges, or roughly 3.7× the fixed cost. Per-job compute is the measured transcode time (Table III) multiplied by the per-second task price, plus up to one minute of billing during scale-in. Because `terraform destroy` removes every resource, the hourly cost between sessions is zero, and the only remaining charge is for the state bucket.

VI. Discussion and Limitations

Spot interruption vs. job length. A Spot interruption gives a 120-second notice, and the worker finishes its current job on `SIGTERM`. A job that needs longer than the remaining notice is killed, and its message reappears after the visibility timeout. That retry counts toward `maxReceiveCount`, so a valid but long video that suffers repeated interruptions could end up in the DLQ. Extending the message's visibility periodically during long jobs (a heartbeat) and keeping the receive count separate from infrastructure retries would remove this risk.

Duplicate concurrent processing. The duplicate skip only applies to items that are already `ready`. A redelivery while a first attempt is still running (for example, after a visibility timeout on a very long transcode) leads to two workers processing the same job. Deterministic output keys make the result correct but waste compute. A conditional update that claims the item, or a visibility heartbeat, would prevent this.

Public subnets. Placing tasks in public subnets removes the NAT gateway cost. Security then rests entirely on security groups, which allow no inbound traffic to workers and only load-balancer traffic to the API. The trade-off is that tasks have public IPs and unrestricted egress. VPC gateway endpoints for S3 and DynamoDB, which have no hourly charge, would keep most data traffic on the AWS network.

Streaming format. Progressive MP4 with manual resolution selection is simple and CDN-friendly, but it does not adapt to changing bandwidth. A fixed CRF ladder also ignores content complexity [8].

OIDC subject pattern. To accept both the standard subject claim and the ID-annotated form some accounts emit, the trust policy also matches `repo:<owner>*<repo>*:*`. Because the wildcard follows the owner name, it also matches repositories owned by any account whose name *starts with* the owner's name. The fix is to match the immutable numeric repository and owner IDs that the annotated claim contains.

Edge HTTP to origin. CloudFront terminates HTTPS, but the hop to the load balancer is plain HTTP. Adding an HTTPS listener with a managed certificate would require a custom domain.

TABLE III
WORKER TRANSCODING TIME IN SECONDS (PRODUCTION IMAGE, 1080P30 INPUT, $n = 3$)

	1 vCPU, 2 GB (deployed task size)					2 vCPU, 2 GB	
Clip (s)	720p	480p	Thumb	Total (mean ± sd)	Total / clip	Total (mean ± sd)	Speedup
15	12.6	9.8	0.6	23.1 ± 0.8	1.54	8.6 ± 0.4	2.68×
30	25.1	18.5	0.7	44.3 ± 1.9	1.48	16.8 ± 0.7	2.63×
60	51.0	37.6	0.7	89.4 ± 4.2	1.49	33.5 ± 1.1	2.67×
120	105.9	77.0	0.7	183.6 ± 6.1	1.53	69.7 ± 1.9	2.63×

TABLE IV
OUTPUT SIZE IN MB (AVERAGE BITRATE IN KB/S)

Clip (s)	Input	720p	480p	Reduction
15	15.5	1.50 (798)	0.67 (359)	86.0%
30	31.0	3.05 (813)	1.36 (364)	85.8%
60	61.7	6.14 (819)	2.73 (365)	85.6%
120	122.6	12.21 (814)	5.46 (364)	85.6%

TABLE V
LIVE DEPLOYMENT MEASUREMENTS (AP-SOUTH-1)

Metric	Value
`terraform apply`, empty account	≈9 min
`terraform destroy`	≈7 min
CI/CD run, full rebuild	≈8 min
Upload to `processing`, idle (0 workers)	≈120 s
Upload to `processing`, warm worker	≈2 s
Upload to `ready`, 30 s clip, idle	≈170 s
Circuit-breaker rollback detected after	≈6 min

A. Threats to Validity

The benchmark enforced the task's resource limits with Docker CPU and memory quotas rather than running on Fargate itself. Fargate vCPUs are hyperthreads on shared hosts, and per-core encoder throughput varies with the underlying processor. The absolute times in Table III should therefore be read as indicative, while the linear scaling with duration and the split between renditions should carry over. The 1- vs. 2-vCPU ratio is the least portable result, because it depends on how the CPU limit is enforced and how many cores FFmpeg sees. Inputs are synthetic. Encoding time and output size both depend on content complexity, and real camera footage with fast motion or fine texture can take longer and produce larger outputs than the test pattern. The size reductions also depend on the 8 Mb/s source bitrate we chose. Measurements on the live deployment (Table V) are needed to confirm the absolute numbers.

VII. CONCLUSION AND FUTURE WORK

StreamForge shows that a video pipeline can combine event-driven elasticity, bounded failure cost, least-privilege access and verifiable automated delivery while keeping its fixed footprint to one load balancer and one small API task. Three design choices do most of the work: presigned uploads keep media off the API tier, queue depth drives scale-to-zero Spot workers, and a single CDN distribution fronts every origin.

TABLE VI
FIXED VS. PER-JOB COST (US-EAST-1 LIST PRICES)

Item	Cost
Fixed while deployed	
ALB (base, excl. LCU)	$0.0225/h
API task (0.25 vCPU, 0.5 GB, on-demand)	$0.0123/h
Worker fleet when idle	$0
NAT gateway	*not deployed*
Avoided by design	
1 NAT gateway (base, excl. data)	$0.045/h
1 always-on worker (1 vCPU, 2 GB, on-demand)	$0.0494/h
Per job	
Worker, on-demand, per minute of 1080p input	$0.00126
Worker, Spot (70% off), per minute of input	$0.00038

Future work includes segmented HLS/DASH output for adaptive streaming [9], visibility heartbeats and conditional claims to remove the issues described in Section VI, per-title ladders, ARM64 worker images (which Fargate supports at a lower price), and automatic captioning triggered by the `ready` transition.

REFERENCES

[1] Cisco Systems, "Cisco Visual Networking Index: Forecast and Trends, 2017–2022," White Paper, 2018.

[2] E. Jonas *et al.*, "Cloud programming simplified: A Berkeley view on serverless computing," Univ. California, Berkeley, Tech. Rep. UCB/EECS-2019-3, 2019.

[3] Amazon Web Services, "Lambda quotas," AWS Lambda Developer Guide. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html

[4] S. Fouladi *et al.*, "Encoding, fast and slow: Low-latency video processing using thousands of tiny threads," in *Proc. 14th USENIX Symp. Networked Syst. Design Implementation (NSDI)*, 2017, pp. 363–376.

[5] L. Ao, L. Izhikevich, G. M. Voelker, and G. Porter, "Sprocket: A serverless video processing framework," in *Proc. ACM Symp. Cloud Computing (SoCC)*, 2018, pp. 263–274.

[6] FFmpeg Developers, "FFmpeg." [Online]. Available: https://ffmpeg.org

[7] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 560–576, Jul. 2003.

[8] A. Aaron, Z. Li, M. Manohara, J. De Cock, and D. Ronca, "Per-title encode optimization," Netflix Technology Blog, Dec. 2015.

[9] T. Stockhammer, "Dynamic adaptive streaming over HTTP: Standards and design principles," in *Proc. 2nd ACM Conf. Multimedia Syst. (MMSys)*, 2011, pp. 133–144.

[10] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.

[11] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.

[12] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd ed. Sebastopol, CA, USA: O'Reilly, 2020.

[13] M. Souppaya, J. Morello, and K. Scarfone, “Application container security guide,” NIST, Gaithersburg, MD, USA, Special Publication 800-190, 2017.
[14] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.
[15] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID Connect Core 1.0,” OpenID Foundation, 2014.
[16] Amazon Web Services, “AWS Fargate pricing.” [Online]. Available: https://aws.amazon.com/fargate/pricing/
[17] Amazon Web Services, “Elastic Load Balancing pricing.” [Online]. Available: https://aws.amazon.com/elasticloadbalancing/pricing/
[18] Amazon Web Services, “Amazon VPC pricing.” [Online]. Available: https://aws.amazon.com/vpc/pricing/
[19] K. Dhonde, K. Jhalak, S. Doshi, S. Kamat, and R. Varshney, “StreamForge source repository.” [Online]. Available: https://github.com/kartikadhonde/cc-proj